

314707003_16.18.R

CDH

2026-05-27

```
rm(list=ls())  
library(lmtest)
```

```
## Warning: 套件 'lmtest' 是用 R 版本 4.5.3 來建造的
```

```
## 載入需要的套件：zoo
```

```
## Warning: 套件 'zoo' 是用 R 版本 4.5.3 來建造的
```

```
##  
## 載入套件：'zoo'
```

```
## 下列物件被遮斷自 'package:base':  
##  
##      as.Date, as.Date.numeric
```

```
library(sandwich)
```

```
## Warning: 套件 'sandwich' 是用 R 版本 4.5.3 來建造的
```

```
# to https://github.com/ccolonescu/PoEdata/blob/master/data/lasvegas.rda down  
load data
```

```
load("C:/Users/CDH/Downloads/lasvegas.rda")  
df = lasvegas
```

```
#16.18(a)  
lpm <- lm(delinquent ~ lvr + ref + insur + rate +  
          amount + credit + term + arm,  
          data = df)
```

```
# White robust SE  
coeftest(lpm, vcov = vcovHC(lpm, type = "HC1"))
```

```
##  
## t test of coefficients:  
##  
##      Estimate Std. Error t value Pr(>|t|)  
## (Intercept)  0.68849128  0.22850643   3.0130 0.0026525 **
```

```
## lvr          0.00162385  0.00067523   2.4049 0.0163601 *
## ref         -0.05932367  0.02402561  -2.4692 0.0137098 *
## insur        -0.48158489  0.03036944 -15.8575 < 2.2e-16 ***
## rate         0.03437614  0.00981944   3.5008 0.0004845 ***
## amount       0.02376801  0.01445089   1.6447 0.1003398
## credit       -0.00044190  0.00020728  -2.1319 0.0332620 *
## term        -0.01261951  0.00355596  -3.5488 0.0004051 ***
## arm          0.12832392  0.02769325   4.6338 4.07e-06 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
#16.18(b)
logit <- glm(delinquent ~ lvr + ref + insur + rate +
             amount + credit + term + arm,
             data = df,
             family = binomial(link = "logit"))

summary(logit)
```

```
##
## Call:
## glm(formula = delinquent ~ lvr + ref + insur + rate + amount +
##       credit + term + arm, family = binomial(link = "logit"), data = df)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)  1.665303   1.992157   0.836 0.403195
## lvr          0.013519   0.008199   1.649 0.099164 .
## ref         -0.525064   0.230491  -2.278 0.022725 *
## insur        -3.122105   0.216906 -14.394 < 2e-16 ***
## rate         0.308656   0.082771   3.729 0.000192 ***
## amount       0.217606   0.113769   1.913 0.055786 .
## credit       -0.003564   0.001933  -1.844 0.065232 .
## term        -0.133037   0.034597  -3.845 0.000120 ***
## arm          1.405649   0.371355   3.785 0.000154 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 998.03  on 999  degrees of freedom
## Residual deviance: 666.29  on 991  degrees of freedom
## AIC: 684.29
##
## Number of Fisher Scoring iterations: 5
```

```
# robust SE
coeftest(logit, vcov = vcovHC(logit, type = "HC1"))
```

```
##
## z test of coefficients:
##
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)  1.6653028  2.1160162   0.7870 0.431282
## lvr          0.0135195  0.0075284   1.7958 0.072527 .
```

```
## ref          -0.5250637  0.2355562  -2.2290  0.025811 *
## insur        -3.1221053  0.2204154 -14.1646 < 2.2e-16 ***
## rate         0.3086558  0.0947648   3.2571  0.001126 **
## amount       0.2176059  0.1247233   1.7447  0.081035 .
## credit       -0.0035644  0.0019702  -1.8091  0.070431 .
## term        -0.1330370  0.0315981  -4.2103  2.550e-05 ***
## arm          1.4056486  0.3492045   4.0253  5.691e-05 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
#16.18(c)
obs <- df[c(500, 1000), ]

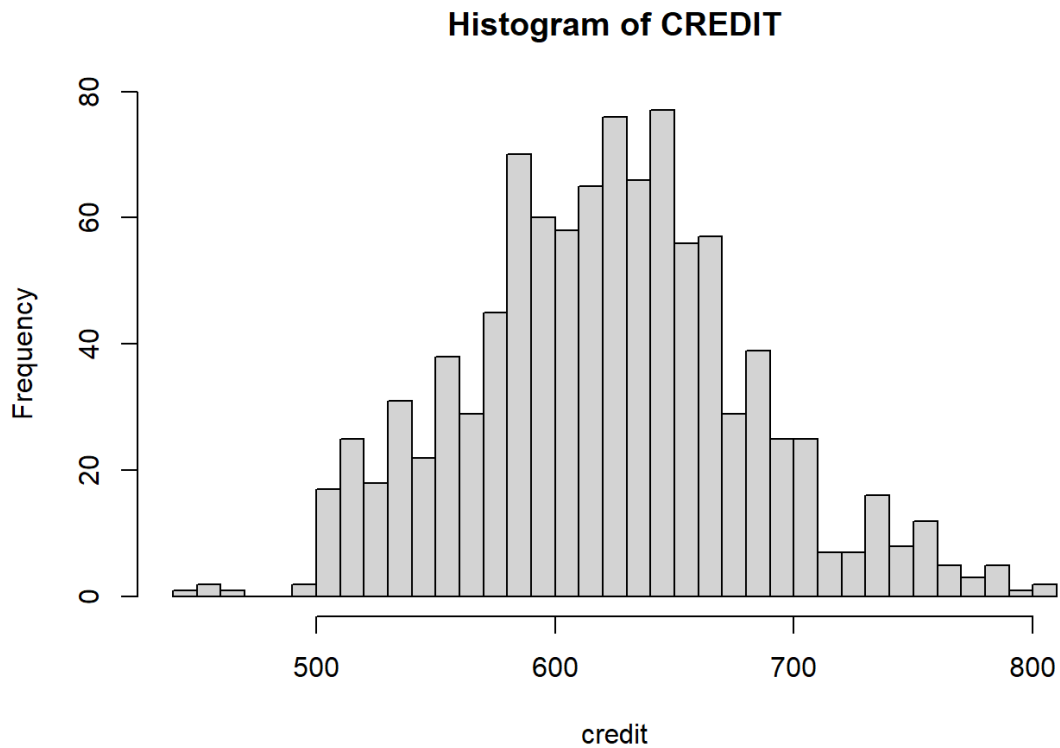
# LPM prediction
pred_lpm <- predict(lpm,
                    newdata = obs)

# Logit prediction
pred_logit <- predict(logit,
                     newdata = obs,
                     type = "response")

cbind(obs,
      pred_lpm,
      pred_logit)
```

```
##      lvr ref insur  rate amount credit term arm delinquent  pred_lpm
## 500  70.0  1     1 10.95  0.854   509  30  1           0 0.1827828
## 1000 88.2  1     0  7.65  2.910   624  30  1           0 0.5785296
##      pred_logit
## 500    0.1335848
## 1000    0.6265535
```

```
#16.18(d)
hist(df$credit,
     main = "Histogram of CREDIT",
     xlab = "credit",
     breaks = 30)
```



```
new_credit <- data.frame(
  lvr = 80,
  ref = 0,
  insur = 0,
  rate = 8,
  amount = 2.5,
  credit = c(500, 600, 700),
  term = 30,
  arm = 0
)

# LPM
prob_lpm_credit <- predict(lpm,
                          newdata = new_credit)

# Logit
prob_logit_credit <- predict(logit,
                             newdata = new_credit,
                             type = "response")

cbind(new_credit,
      prob_lpm_credit,
      prob_logit_credit)
```

```
##   lvr ref insur rate amount credit term arm prob_lpm_credit prob_logit_cre
## 1  80  0   0    8   2.5   500   30  0         0.5532956         0.4967
## 2  80  0   0    8   2.5   600   30  0         0.5532956         0.4967
## 3  80  0   0    8   2.5   700   30  0         0.5532956         0.4967
```

## 2	80	0	0	8	2.5	600	30	0	0.5091060	0.4086
308										
## 3	80	0	0	8	2.5	700	30	0	0.4649165	0.3260
580										

```
#16.18(e)
# LPM marginal effect
me_lpm_credit <- coef(lpm)["credit"]

# Logit marginal effect
p_credit <- predict(logit,
                    newdata = new_credit,
                    type = "response")

me_logit_credit <- coef(logit)["credit"] *
  p_credit *
  (1 - p_credit)

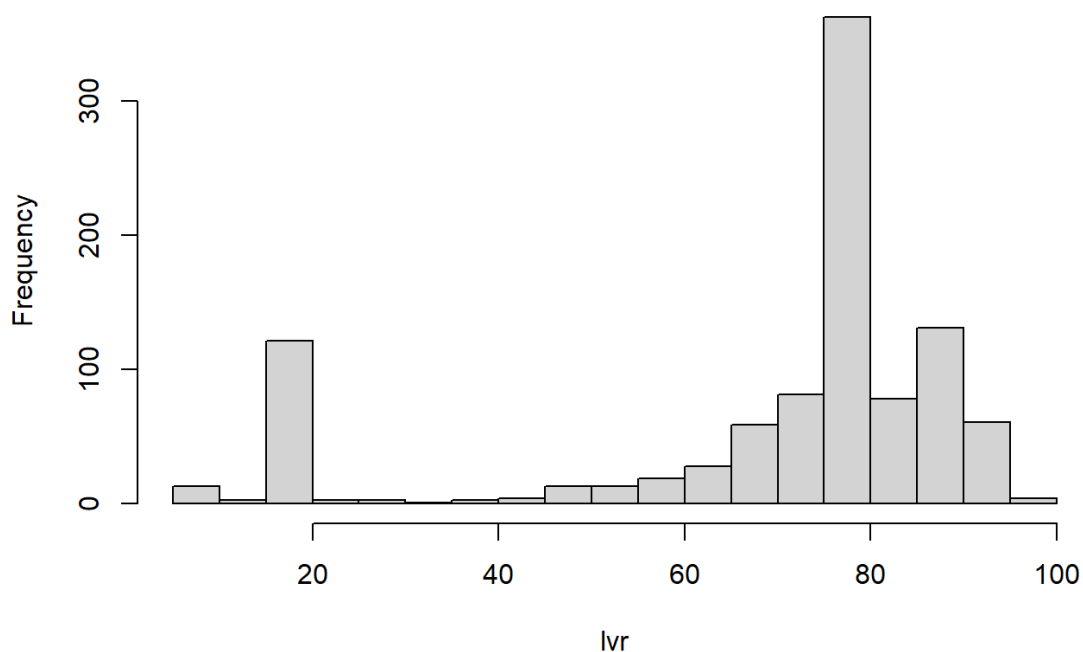
data.frame(
  credit = new_credit$credit,
  me_lpm = me_lpm_credit,
  me_logit = me_logit_credit
)
```

```
## Warning in data.frame(credit = new_credit$credit, me_lpm = me_lpm_credit,
:
## 短變數裡找到了列名稱，因此不用
```

##	credit	me_lpm	me_logit
## 1	500	-0.0004418953	-0.0008910620
## 2	600	-0.0004418953	-0.0008613439
## 3	700	-0.0004418953	-0.0007832567

```
#16.18(f)
hist(df$lvr,
     main = "Histogram of LVR",
     xlab = "lvr",
     breaks = 30)
```

Histogram of LVR



```
new_lvr <- data.frame(
  lvr = c(20, 80),
  ref = 0,
  insur = 0,
  rate = 8,
  amount = 2.5,
  credit = 600,
  term = 30,
  arm = 0
)

# LPM
prob_lpm_lvr <- predict(lpm,
                        newdata = new_lvr)

# Logit
prob_logit_lvr <- predict(logit,
                          newdata = new_lvr,
                          type = "response")

cbind(new_lvr,
      prob_lpm_lvr,
      prob_logit_lvr)
```

##	lvr	ref	insur	rate	amount	credit	term	arm	prob_lpm_lvr	prob_logit_lvr
## 1	20	0	0	8	2.5	600	30	0	0.4116749	0.2349087
## 2	80	0	0	8	2.5	600	30	0	0.5091060	0.4086308

```
#16.18(g)
# 預測值
p_lpm_all <- predict(lpm)

p_logit_all <- predict(logit,
                      type = "response")

# classification
yhat_lpm <- ifelse(p_lpm_all >= 0.5, 1, 0)

yhat_logit <- ifelse(p_logit_all >= 0.5, 1, 0)

# accuracy
acc_lpm <- mean(yhat_lpm == df$delinquent)

acc_logit <- mean(yhat_logit == df$delinquent)

acc_lpm
```

```
## [1] 0.858
```

```
acc_logit
```

```
## [1] 0.854
```

```
# confusion matrix
table(actual = df$delinquent,
      pred_lpm = yhat_lpm)
```

```
##      pred_lpm
## actual    0    1
##      0 727  74
##      1  68 131
```

```
table(actual = df$delinquent,
      pred_logit = yhat_logit)
```

```
##      pred_logit
## actual    0    1
##      0 736  65
##      1  81 118
```

```
#16.18(h)
# training data
train <- df[1:500, ]

# testing data
test <- df[501:1000, ]

# train logit
```



```

logit_train <- glm(delinquent ~ lvr + ref + insur + rate +
                  amount + credit + term + arm,
                  data = train,
                  family = binomial(link = "logit"))

# prediction on test data
p_test <- predict(logit_train,
                  newdata = test,
                  type = "response")

#####
# threshold = 0.5
#####

approve_05 <- ifelse(p_test < 0.5, 1, 0)

repay <- ifelse(test$delinquent == 0, 1, 0)

acc_05 <- mean(approve_05 == repay)

acc_05

```

```
## [1] 0.88
```

```

#####
# search for best threshold
#####

thresholds <- seq(0.01, 0.99, by = 0.01)

acc <- sapply(thresholds, function(cut){
  approve <- ifelse(p_test < cut, 1, 0)
  mean(approve == repay)
})

best_cut <- thresholds[which.max(acc)]

best_acc <- max(acc)

best_cut

```

```
## [1] 0.49
```

```
best_acc
```

```
## [1] 0.884
```

```

# plot
plot(thresholds,
      acc,

```

```
type = "l",  
xlab = "threshold",  
ylab = "accuracy",  
main = "Threshold Selection")
```

```
abline(v = best_cut, lty = 2)
```

